

Problèmes P/NP et algorithmique avancée

Dr. Zakaria Sawadogo.

zakaria.sawado@gmail.com

Problèmes P/NP et algorithmique avancée

- 1 Généralités
- 2 Classification des problèmes
- 3 Pistes de solution
- 4 Applications du Diviser pour régner
- 5 Applications des algorithmes gloutons
- 6 Applications de la programmation dynamique

- 1 Généralités
- 2 Classification des problèmes
- 3 Pistes de solution
- 4 Applications du Diviser pour régner
- 5 Applications des algorithmes gloutons
- 6 Applications de la programmation dynamique

Alan Turing

Qui est Alan Turing ?

Alan Turing est un mathématicien et informaticien anglais connu pour :

- le décryptage des informations d'**Enigma** ;
- la description d'un modèle dit **machine de Turing**

Calculabilité

La calculabilité est la **possibilité de déterminer** si pour une fonction f il existe une **succession d'opérations élémentaires** permettant de connaître sans ambiguïté l'image de n'importe quelle valeur x par f .

Décidabilité

Une propriété est **décidable** s'il y a un **procédé** qui détermine, au bout d'un temps **fini**, si c'est vrai ou faux dans n'importe quel contexte.

Machine de Turing

Description d'une machine de Turing

Une machine de Turing est comme une machine à écrire à ruban sans clavier où :

- le **ruban** est infini et composé de **cases** sur lesquelles sont imprimés des **symboles** d'un ensemble fini connu à l'avance ;
- une **fenêtre** permet à une **tête de lecture** de lire une case du ruban pour que la machine identifie le symbole écrit, le gomme ou le remplace. Le ruban peut se **déplacer** à droite ou à gauche ;
- il existe un nombre fini **d'états internes** qui conditionnent les réactions de la machine aux nouveaux symboles lus sur le ruban. Une **table** décrit, pour chaque état interne, l'action exacte que la machine doit exécuter.

Machine de Turing

Machine de Turing et algorithmique

La machine de Turing est comme un algorithme car :

- l'algorithme consiste en un ensemble fini d'instructions simples et précises, décrites avec un nombre limité de symboles ;
- l'algorithme détermine toujours le résultat en un nombre fini d'étapes ;
- l'algorithme peut être suivi par un humain avec seulement du papier et un crayon ;
- l'exécution de l'algorithme ne requiert pas d'intelligence humaine sauf celle pour comprendre et exécuter les instructions.

La machine était vue comme un procédé mécanique où des agents de calcul seraient bornés à suivre et exécuter des instructions.

Machine de Turing

Liens entre machine de Turing, calculabilité et décidabilité

Les travaux de Turing ont montré :

- qu'une fonction est **calculable** si elle est programmable sur machine de Turing ;
- que résoudre la **décidabilité** équivaut à pronostiquer la terminaison d'un programme sur une machine de Turing ;
- que **la décidabilité est équivalente à la calculabilité**. En effet une propriété mathématique $P(a)$ est une fonction vers l'ensemble $\{\text{vrai}, \text{faux}\}$ et pour toute fonction $y = f(x)$, on peut associer la propriété $P(x, y)$ à VRAI si $y = f(x)$ et à FAUX sinon.

- 1 Généralités
- 2 Classification des problèmes
 - P et NP
 - Complétude d'un problème
 - Exemple de problèmes concrets
- 3 Pistes de solution
- 4 Applications du Diviser pour régner
- 5 Applications des algorithmes gloutons
- 6 Applications de la programmation dynamique

Classes P et NP

Machine déterministe VS Machine non déterministe

- Une machine **déterministe** est la machine de Turing telle que présentée. Une telle machine fait toujours un seul calcul à la fois. Ce calcul est fait d'étapes élémentaires où pour un état donné de la mémoire, l'action élémentaire effectuée sera toujours la même.
- Une machine **non-déterministe** est comme une machine de Turing MAIS qui, à chaque étape de calcul, peut effectuer un choix entre plusieurs actions. Si l'un des choix est bon, on considère que c'est ce choix qui est fait.
- **Les machines non-déterministes sont perçues comme plus puissantes que les machines déterministes.** Mais la difficulté est la question de fabrication de la machine (l'ordinateur à ADN).

Classes P et NP

Problème P

Un problème **P** peut être résolu par un algorithme déterministe en un temps polynomial. Les problèmes **P** constituent l'ensemble des problèmes facilement solubles.

Problème P

- **Problème** : Déterminer si un graphe est connexe, i.e. qu'il existe un chemin pour joindre 2 noeuds quelconque du graphe.
- **Exemple de solution** : Parcours en profondeur du graphe pour voir si on atteint tous les noeuds.

Classes P et NP

Problème NP

Un problème **NP** (Non-déterministe Polynomial et non pas Non polynomial) est un problème qui peut être résolu en énumérant l'ensemble des solutions possibles et en testant une avec un algorithme polynomial.

Problème NP

- **Problème** : Recherche de cycle hamiltonien dans un graphe, i.e. un chemin qui part d'un noeud, passe par tous les autres et revient au noeud de départ.
- **Exemple de solution** : Génération des cycles possibles en temps exponentiel puis test de cycle en temps polynomial.

Classes P et NP

Liens entre P et NP

- Une erreur classique est de penser que **NP** signifie **Non Polynomiale**, ce qui est faux pour au moins 2 raisons :
 - **P et NP ne sont pas complémentaires** : P est même totalement inclus dans NP dont il est un cas particulier ;
 - il existe des problèmes qui ne sont **ni dans P ni dans NP**.
- $P \in NP$ mais la question de savoir si $NP \in P$ (ce qui impliquerait $P = NP$) est VRAI ou FAUX reste l'un des (6) problèmes du millénaire ouverts. La résolution de ce problème est dotée d'un prix de 1 000 000 \$ US par l'institut mathématiques Clay.

Complétude

Problème (P/NP)-difficile

Un problème est (P/NP)-difficile s'il est **au moins aussi dur** que tous les problèmes dans (P/NP).

Problème (P/NP)-Complet

Un problème est (P/NP)-Complet :

- s'il est dans (P/NP) ;
- et s'il est (P/NP)-difficile.

NP-Difficile VS NP-Complet

La différence est qu'un **problème NP-difficile n'est pas forcément dans NP** soit parce que ce n'est pas prouvé ou parce que même tester une solution ne peut se faire en temps polynomial.

Complétude

Problème NP-Complet : problème de la satisfaisabilité **SAT**

Ce problème se décrit comme suit :

- En entrée on a n variables booléennes $x_1, x_2, \dots, x_n$;
- On veut savoir si une proposition logique de ces variables $(x_{i1} \vee x_{j1} \dots \vee x_{k1}) \wedge (x_{i2} \vee x_{j2} \dots \vee x_{k2}) \wedge \dots \wedge (x_{im} \vee x_{jm} \dots \vee x_{km})$ est vraie en assignant aux x_{it} des valeurs des x_i (0 ou 1).

Importance du problème **SAT** en cryptographie

Une fonction de hachage est une conjonction de clauses normales, dont les variables booléennes correspondent aux valeurs possibles des données d'entrée de la fonction de hachage, et chacun des bits de résultat devra répondre à un test booléen d'égalité avec les bits de données d'un bloc de données d'entrées quelconque.

Complétude

Problème NP-Complet : problème du voyageur de commerce

Ce problème se décrit comme suit :

- On a n points (villes) $x_1, x_2, \dots, x_n$ avec des distances entre elles de d_{ij} ;
- On veut trouver un chemin de longueur minimale qui passe **exactement une fois par chaque point et revienne au point de départ**.

Importance du problème du voyageur de commerce

Comme exemple de problèmes similaires on peut citer l'optimisation de parcours de bus ou l'optimisation (temps ou coût) du déplacement (couteux) d'équipements comme un télescope.

Problème SAT (SATisfiability problem) et cryptographie

SAT

Le problème SAT consiste à décider si pour une formule de la logique propositionnelle (conjonction de propositions de variables booléennes) il existe une assignation des variables qui rend la formule vraie.

Lien avec la cryptographie

Les fonctions de hachages sont utilisées pour authentifier et signer des messages. En algorithmique, on peut voir une fonction de hachage comme une fonction déterministe $h : B^* \leftarrow B^k$ (où $B = \{0,1\}$) pour un $k \in \mathbb{N}$ fixé. C'est à dire que h transforme n'importe quel *bitstring* (élément du message B^*) de longueur arbitraire en un *bitstring* de taille fixée.

Problème SAT (SATisfiability problem) et cryptographie

Lien avec la cryptographie

Une fonction de hachage ne peut pas être injective (à cause de la *cardinalité*). Cependant, elle respecte ces propriétés :

- **résistance de la pré-image** : il est impossible par calcul (en termes algorithmique, la complexité est exponentielle) de calculer x en connaissant $h(x)$. On ne peut pas déterminer le message à partir du *digest* ;
- **résistance de la seconde pré-image** : il est impossible par calcul de calculer x' en connaissant x tel que $h(x) = h(x')$. Cela garantit l'impossibilité pour un signataire de changer le contrat en c' tel que $h(c) = h(c')$ et de prouver son authenticité par $sign(h(c)) = sign(h(c'))$.

Problème SAT (SATisfiability problem) et cryptographie

Cryptanalyse logique

- La cryptanalyse logique consiste à mesurer la sécurité d'une primitive cryptographique pour montrer qu'une garantie de sécurité attendue n'est pas vérifiée. Par exemple en faisant un algorithme trouvant des pré-images en temps sous-exponentiel.
- Le principe consiste à casser une primitive en réduisant le problème sensé être de complexité exponentielle en une formule de calcul propositionnel dont on teste ensuite la *satisfaisabilité* en utilisant un SAT-solveur. Dans le cas positif, on doit ensuite pouvoir extraire de la valuation une solution au problème.

Problème SAT (SATisfiability problem) et cryptographie

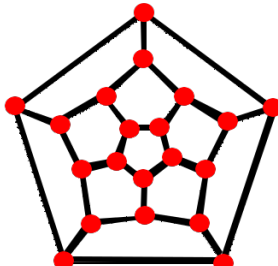
Attaque de fonction de hachage = Problème SAT

- On peut encoder x (message inconnu) tel que $h(x) = d$ pour $d \in B^{128}$ fixé dans une formule (forme normale conjonctive).
- Cette formule est nécessairement *satisfiable* et si un SAT-solveur nous donne une valuation possible alors on peut en extraire un m tel que $h(m) = d$. Et si on peut mener à bien cette tâche alors on aura cassé (nous même) la fonction h en mettant en place une attaque sur la propriété de la résistance de la pré-image.
- Néanmoins une telle attaque, même pour un nombre d'étapes inférieur à 64 est tout de même significative (lent) : souvent les attaques sont incrémentales et les cryptographes s'inquiètent bien avant que l'on casse la primitive dans son intégralité.

Problème du voyageur de commerce

Problématique

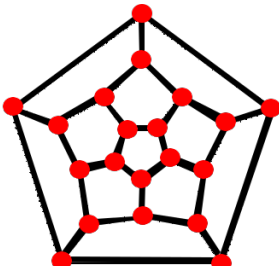
Imaginez un commercial à qui le patron a donné une liste de 20 villes à démarcher avec la contrainte de passer une seule fois par ville. Renseignements pris, toutes les villes ne sont pas reliées entre elles : les cars n'assurent que certaines liaisons. Quel sera son trajet ?



Problème du voyageur de commerce

Comment on résout un problème NP ?

Passer une fois par chaque ville en utilisant les arêtes et finir au point de départ est similaire à la définition d'un **cycle hamiltonien** dans un graphe. Or la recherche d'un cycle hamiltonien est NP-complet : on ne connaît pas d'algorithme plus efficace que la **brute force**.



Problème du voyageur de commerce

Piste de solution

- NP-complet ne signifie pas forcément *algorithme complexe (difficile à écrire)* ! D'ailleurs quand on veut résoudre un problème NP-complet par un algorithme *brute-force*, l'algorithme est généralement simple mais seulement extrêmement long en termes de nombre d'opérations / temps de calcul.
- Dans notre cas, on peut résoudre le problème en une quinzaine de lignes avec une méthode récursive. Mieux encore, on peut travailler avec un simple pointeur, ce qui va énormément accélérer les choses.

Problème du voyageur de commerce

Définition de la structure de données (graphe)

Soit un tableau dont les clés sont les nœuds, et les valeurs les nœuds accessibles depuis le nœud-clé. Par exemple :

1 => 2,5

2 => 1,3,5

3 => 2,4

4 => 3,5,6

5 => 1,2,4

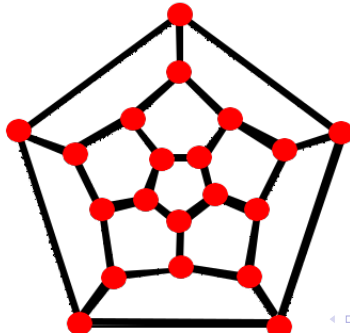
6 => 4

Dessiner le graphe correspondant. Peut-on y avoir un cycle hamiltonien ? Proposer une modification pour en avoir.

Problème du voyageur de commerce

Structure de données

Utiliser le principe précédent pour coder la structure de données correspondant au graphe :



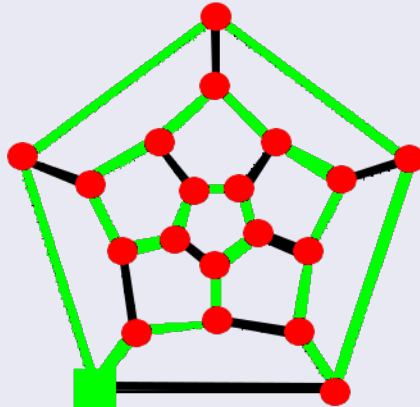
Problème du voyageur de commerce

Algorithme

- Choisir un point de départ, par exemple l'élément d'indice 1 ;
- On récupère la liste des destinations disponibles depuis 1 ;
- On supprime 1 de la liste des nœuds pour ne pas y revenir ;
- On rappelle la fonction pour toute destination de 1 (récursion) ;
- La première récursion se fait sur l'indice 1 des destinations de l'élément supprimé. On récupère la liste des itinéraires de cet indice 1 et on y cherche un chemin. On continue récursivement ;
- On s'arrête quand on a parcouru autant de nœuds qu'il y en a dans le graphe et que le dernier nœud a un chemin d'accès vers l'élément 1 (pour boucler). Sinon, on continue de chercher des itinéraires ...

Problème du voyageur de commerce

Illustration d'un chemin optimal



Algorithme

```
trouve(nods:Tableau,cur:Entier à 1,test:Entier à 0):Booléen
DEBUT
//Récupérer la liste des itinéraires depuis le noeud Actu
itns=nods[cur];
//Supprimer ce noeud de la liste (pour qu'on ne le reprenne pas)
nods[cur]=null;
//Examiner chacun des itinéraires depuis le noeud courant
Pour chaque (itn:Entier dans itns) Faire
Si ((test==nb ET itn==1) OU (nods[itn]!=null ET trouve(nods,itn,test+1)))
trace("Trouvé : " + Actuel);
Retourner Vrai;
FIN Si
FIN Pour
//Sinon, aucun chemin hamiltonien: réinitialiser le tableau, puis revenir
nods[cur]=itns;
Retourner Faux;
FIN
```

Problèmes P/NP et algorithmique avancée

- 1 Généralités
- 2 Classification des problèmes
- 3 Pistes de solution**
- 4 Applications du Diviser pour régner
- 5 Applications des algorithmes gloutons
- 6 Applications de la programmation dynamique

Réduction

Réduction

On considère 2 problèmes P et P' . On appelle **réduction** de P' vers P un algorithme transformant toute instance de P' en une instance de P . L'idée est que, si l'on a un algorithme pour résoudre P , on sait aussi résoudre P' . P est donc au moins aussi difficile à résoudre que P' .

Remarques

- Si un problème de complexité $\mathbf{C}$ se réduit en $P \Rightarrow P$ est $\mathbf{C}$ -difficile ;
- Implicitement la **réduction** de problèmes NP-difficiles (algorithme de passage de P' à P) doit être de complexité polynomiale ;
- La réduction amène du **problème à classer vers un problème classé.**

Réduction

Réduction : recherche de cycle hamiltonien dans un graphe orienté

- **Le problème est dans NP** : sur une machine non-déterministe un algorithme peut énoncer tous les cycles puis en déterminer le plus court. Or (**algorithme de réduction**) :
 - la recherche de cycle hamiltonien d'un graphe non-orienté est NP-difficile ;
 - un graphe non-orienté se transforme en graphe orienté en doublant chaque arête ;
 - donc on peut ramener le problème de recherche de cycle hamiltonien d'un graphe non-orienté au problème de recherche de cycle hamiltonien d'un graphe orienté ;
 - la recherche de cycle hamiltonien d'un graphe orienté est donc NP-difficile.
- Le problème étant dans NP et NP-difficile, il est NP-complet.

Réduction

Pistes de résolutions : partons d'un problème

- Une organisation crapuleuse a une combine pour détourner l'argent dédié aux lignes de bus scolaire au niveau de directions provinciales, et faire payer les provinces voisines.
- Cependant, il n'est pas possible d'utiliser cette technique au-delà d'un montant identique dans toutes les provinces. De plus, on ne peut pas l'utiliser dans deux provinces voisines sinon les chauffeurs ne seraient plus payés.
- On veut calculer la combine maximale pour contrecarrer. Pour cela il faut trouver le plus grand ensemble de provinces sans deux provinces voisines.

Réduction

Pistes de résolutions : Modélisation

Pour modéliser ce genre de problème, on utilise le graphe. Un graphe est un ensemble de sommets (provinces) reliés ou non par des arêtes (provinces voisines ou non). Par exemple la Comoé peut être s_6 et le Houet s_{10} avec (s_6, s_{10}) une arête. On formalise ensuite le problème comme suit :

- E est l'ensemble des arêtes ;
- V est l'ensemble des sommets ;
- $MAX(X), X \in V | \forall i, j \in X, (i, j) \notin E$

C'est un problème NP-difficile appelé **Maximum Independent Set** (MAX IS) que l'optimisation combinatoire étudie et essaie de résoudre.

Réduction

Première piste : Abandon

Ce n'est pas intellectuellement acceptable, mais quand on n'arrive pas à trouver la solution, on est contraint d'abandonner. Cela peut arriver :

- parce que les règles du problème ont changé ;
- parce qu'on ne peut pas y arriver avec la contrainte de temps imposée ;
- parce qu'on ne sait pas résoudre le problème, même sans contrainte de temps, car le problème est trop difficile. Par exemple parce qu'un nombre infini de stratégies seraient possibles.

Beaucoup de problèmes NP-difficiles sont abandonnés parce qu'on ne sait pas les résoudre.

Réduction

Deuxième piste : recherche exhaustive

- Comme on cherche un sous-ensemble de provinces, on peut toujours regarder tous les sous-ensembles possibles. Il suffira alors d'éliminer ceux qui contiennent des voisins, et prendre le plus grand de ceux qui restent.
- Si on applique cela aux 45 provinces, on aura 2^{45} (35184372088832) sous-ensembles à analyser (par ordinateur). C'est faisable si on a le temps et les conditions (délestage = reprise à 0) ;
- Lorsque cela est possible, on a plus d'informations que dans d'autres approches. Par exemple on peut savoir si la solution est unique, et si ce n'est pas le cas, on peut avoir toutes les solutions.

Réduction

Troisième piste : Algorithme glouton

- Un algorithme glouton est un algorithme dans lequel on cherche, à chaque étape, à faire les choses le mieux possible, mais sans aucun plan d'ensemble. Il fonctionne très rapidement mais n'est pas efficace.
- Dans certaines situations il donne la meilleure solution. Par exemple, pour rendre la monnaie, si on choisit toujours la plus grosse pièce possible, on rend toujours le nombre minimum de pièces.

Réduction

Algorithme glouton pour MAX IS

Entrée : graphe

Sortie : ensemble stable

DEBUT

Tant que **V non vide** faire

Ajouter un sommet dans X

Retirer les voisins de ce sommet de V.

FIN

Réduction

Quatrième piste : Heuristique

Dans les cas où la solution de l'algorithme glouton n'est pas satisfaisante, on a parfois des idées pour améliorer la situation une fois qu'il est terminé.

Réduction

Heuristique pour MAX IS

Entrée : graphe

Sortie : ensemble stable

DEBUT

Appliquer l'algorithme glouton et obtenir une solution S

Faire un grand nombre de fois

Échanger s_i de X avec s_k de V

Si on peut rajouter un autre sommet alors

Enregistrer l'échange

Rajouter l'autre sommet

Sinon Annuler l'échange

Fin Si

Fin faire

FIN

Réduction

Cinquième piste : algorithme approché

Comme avec une heuristique, on ne cherche plus la meilleure solution, mais on veut des garanties qu'on n'aura pas une solution *trop mauvaise*. Concrètement, cela se fait en fixant un rapport d'approximation (par exemple 80%), et en exigeant ensuite d'avoir au moins ce rapport entre la solution que l'on trouve et la meilleure possible (si la meilleure solution est 20 provinces, on en aura au moins 16).

Réduction

Algorithme approché à 50% pour MAX IS

Entrée : graphe

Sortie : ensemble stable approché

DEBUT

Séparer le graphe en deux parties (par exemple de 1 à 23 et 24 à 45)

Calculer le MAX IS sur chacune des deux parties

Donner la meilleure des deux solutions

FIN

Problèmes P/NP et algorithmique avancée

- 1 Généralités
- 2 Classification des problèmes
- 3 Pistes de solution
- 4 Applications du Diviser pour régner
 - Multiplications de matrices
 - Multiplications de polynômes
 - Tris basés sur "Diviser pour régner"
- 5 Applications des algorithmes gloutons
- 6 Applications de la programmation dynamique

Généralités

Méthode classique

Soit A et B deux matrices $n \times n$, le produit des deux matrices, noté AB , de taille $n \times n$ est :

$$\begin{pmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ \dots & \dots & \dots & \dots \\ c_{n1} & c_{n2} & \dots & c_{nn} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ \dots & \dots & \dots & \dots \\ b_{n1} & b_{n2} & \dots & b_{nn} \end{pmatrix}$$

avec

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{jk} \text{ et } i, j = \overline{1, n}$$

Nombre d'opérations

Il faut alors n^3 multiplications et $n^2 \times (n-1)$ additions pour faire le produit de deux matrices $n \times n$. Soit $T(n) = O(n^3)$.

Multiplication par diviser régner : Algorithme de Strassen

Cas de matrice 2×2

Considérons le produit de 2 matrices 2×2 :

$$\begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

avec

- $c_{11} = a_{11}b_{11} + a_{12}b_{21}$
- $c_{12} = a_{11}b_{21} + a_{12}b_{22}$
- $c_{21} = a_{21}b_{11} + a_{22}b_{21}$
- $c_{22} = a_{21}b_{21} + a_{22}b_{22}$

soit **8 multiplications et 4 additions.**

Multiplication par diviser régner : Algorithme de Strassen

Principe

Volker Strassen propose un moyen de multiplier 2 matrices 2×2 en utilisant que **7 produits** :

- $p_1 = (a_{11} + a_{22})(b_{11} + b_{22})$
- $p_2 = (a_{21} + a_{22})(b_{11})$
- $p_3 = (a_{11})(b_{12} - b_{22})$
- $p_4 = (a_{22})(b_{21} - b_{11})$
- $p_5 = (a_{11} + a_{12})(b_{22})$
- $p_6 = (a_{21} - a_{11})(b_{11} + b_{12})$
- $p_7 = (a_{12} - a_{22})(b_{21} + b_{22})$

tels que :

- $c_{11} = p_1 + p_4 - p_5 + p_7$
- $c_{12} = p_3 + p_5$
- $c_{21} = p_2 + p_4$
- $c_{22} = p_1 - p_2 + p_3 + p_6$

Multiplication par diviser régner : Algorithme de Strassen

Mise en œuvre de l'algorithme de Strassen

Pour faire le produit $C = AB$ de matrices $n \times n$ avec $n = 2^k$, on utilise le diviser régner :

- On divise chaque matrice en 4 sous-matrices de taille $\frac{n}{2} \times \frac{n}{2}$
- On note A_{ij} , B_{ij} et C_{ij} avec $i, j = \overline{1, 2}$ les sous-matrices ainsi générées. On pose ensuite :

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

Multiplication par diviser régner : Algorithme de Strassen

Fonction Strassen(A : Matrice[n][n], B : Matrice[n][n], n) : Matrice[n][n]

Déclarations

C : Matrice[n][n]

Debut

Si (n=1) Alors

Retourner A*B

Sinon

$p1 = \text{Strassen}(a11 + a22, b11 + b22, \frac{n}{2})$

$p2 = \text{Strassen}(a21 + a22, b11, \frac{n}{2})$

$p3 = \text{Strassen}(a11, b12 - b22, \frac{n}{2})$

$p4 = \text{Strassen}(a22, b21 - b11, \frac{n}{2})$

$p5 = \text{Strassen}(a11 + a12, b22, \frac{n}{2})$

$p6 = \text{Strassen}(a21 - a11, b11 + b12, \frac{n}{2})$

$p7 = \text{Strassen}(a12 - a22, b21 + b22, \frac{n}{2})$

$c[1][1] = p1 + p4 - p5 + p7$

$c[1][2] = p3 + p5$

$c[2][1] = p2 + p4$

$c[2][2] = p1 - p2 + p3 + p6$

Retourner C

Multiplication par diviser régner : Algorithme de Strassen

Complexité de l'algorithme de Strassen

$$\begin{aligned}
 T(n) &= 7T\left(\frac{n}{2}\right) + 18\left(\frac{n}{2}\right)^2 \\
 &= 7^2 T\left(\frac{n}{2^2}\right) + 7 * 18\left(\frac{n}{2^2}\right)^2 + 18\left(\frac{n}{2}\right)^2 \\
 &= 7^3 T\left(\frac{n}{2^3}\right) + 7^2 * 18\left(\frac{n}{2^3}\right)^2 + 7 * 18\left(\frac{n}{2^2}\right)^2 + 18\left(\frac{n}{2}\right)^2 \\
 &= \dots \\
 &= 7^k \left(\frac{n}{2^k}\right) + \frac{18n^2}{4} \left(\sum_{i=0}^{k-1} \left(\frac{7}{4}\right)^i\right) \\
 &= O(n^{\log_2 7}) \\
 &\approx O(n^{2.81})
 \end{aligned}$$

Généralités

Intérêt pour la multiplication de polynômes

La multiplications de polynômes intervient dans beaucoup d'algorithmes comme :

- le calcul du **PGCD** ;
- la factorisation en une ou plusieurs variables ;
- la composition de séries formelles ;
- l'interpolation.

Certains problèmes (en cryptologie par exemple) nécessitent de manipuler des polynômes de degré d'ordre 100000 ; ce qui nécessite l'utilisation d'algorithmes rapides.

Généralités

Approche naïve

Soit F et G deux polynômes de degré inférieur ou égal à $n - 1$:

- $F = f_0 + f_1x + f_2x^2 + \dots + f_{n-1}x^{n-1}$
- $G = g_0 + g_1x + g_2x^2 + \dots + g_{n-1}x^{n-1}$

On peut définir formellement le problème comme le calcul des coefficients de $H = F \times G = h_0 + h_1x + \dots + h_{2n-2}x^{2n-2}$

$$H = F \times G = (1 + 4x + x^2) \times (3 + 5x + 2x^2)$$

$$\begin{array}{rcl}
 H & = & 3 \quad +5x \quad +2x^2 \\
 & & \quad +12x \quad +20x^2 \quad +8x^3 \\
 & & \quad \quad +3x^2 \quad +5x^3 \quad +2x^4 \\
 & = & 3 \quad +17x \quad +25x^2 \quad +13x^3 \quad +2x^4
 \end{array}$$

Généralités

Algorithme naïf

$$H = F \times G = \sum_{i=0}^{2n-2} h_i x^i$$

avec

$$h_i = \sum_{j+k=i} f_j g_k$$

Complexité

$$T(n) = O(n^2)$$

Multiplications par algorithme de Karatsuba

Principe

- Considérons F et G deux polynômes de degré 1 :
 - $F = f_0 + f_1 x$
 - $G = g_0 + g_1 x$
- On a alors $H = F \times G = f_0 g_0 + (f_0 g_1 + f_1 g_0)x + f_1 g_1 x^2$
- Si on suit l'algorithme naïf on doit effectuer les 4 multiplications $f_0 g_0, f_0 g_1, f_1 g_0, f_1 g_1$;
- Dans l'approche de **Karatsuba** on part du fait que

$$f_0 g_1 + f_1 g_0 = (f_0 + f_1)(g_0 + g_1) - f_0 g_0 - f_1 g_1$$

pour ramener à 3 multiplications et 4 additions.

Multiplications par algorithme de Karatsuba

Généralisation

- Considérons F et G deux polynômes de degré inférieur ou égal à $n - 1$ ($n = 2^k$). On peut écrire :
 - $F = F^{(0)} + F^{(1)}x^k$
 - $G = G^{(0)} + G^{(1)}x^k$
- avec $F^{(0)}$, $F^{(1)}$, $G^{(0)}$ et $G^{(1)}$ de degré inférieur ou égal à $k - 1$
- $H = F \times G = F^{(0)}G^{(0)} + (F^{(0)}G^{(1)} + F^{(1)}G^{(0)})x^k + F^{(1)}G^{(1)}x^{2k}$.

Complexité : nombre de multiplications

- $T(1) = 1$
- $T(n) = 3T\left(\frac{n}{2}\right) + 4n$
- $T(n) = O(n^{\log_2 3}) \approx O(n^{1.59})$

Multiplications par algorithme de Karatsuba

Algorithme

Fonction Karatsuba(F : Polynôme $n-1$, G : Polynôme $n-1$) : Polynôme $2n-1$

Déclarations

H : Polynôme $n-1$

Debut

Si ($n=1$) Alors

Retourner $F * G$

Sinon

$P1 = \text{Karatsuba}(F^{(0)}, G^{(0)})$

$P2 = \text{Karatsuba}(F^{(1)}, G^{(1)})$

$P3 = F^{(0)} + F^{(1)}$

$P4 = G^{(0)} + G^{(1)}$

$P5 = \text{Karatsuba}(P3, P4)$

$P6 = P5 - P1$

$P7 = P6 - P2$

Retourner $P1 + P7x^{\frac{n}{2}} + P2x^n$

Fin Si

Fin

Tri fusion

Principe

Ce tri utilise la notion de récursivité ; pour trier un tableau de taille N :

- on le subdivise en 2 tableaux de taille $N/2$;
- on trie chacun des 2 tableaux ;
- on fusionne les 2 tableaux de sorte à ce que le tableau final soit trié.

Tri fusion

Illustration du tri par fusion

9	2	7	8	3	5
---	---	---	---	---	---

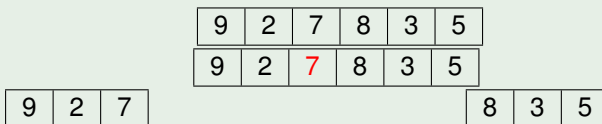
Tri fusion

Illustration du tri par fusion

9	2	7	8	3	5
9	2	7	8	3	5

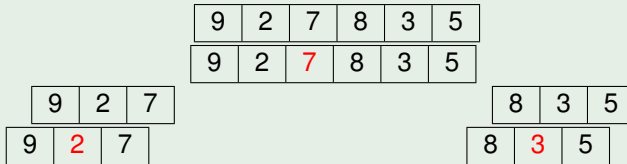
Tri fusion

Illustration du tri par fusion



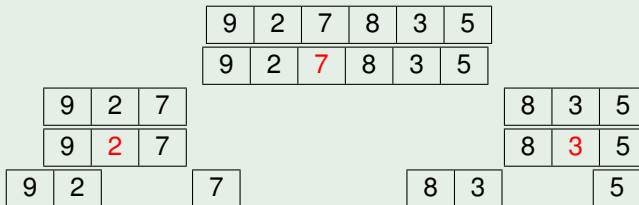
Tri fusion

Illustration du tri par fusion



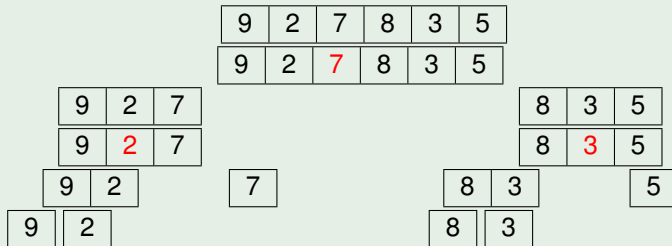
Tri fusion

Illustration du tri par fusion



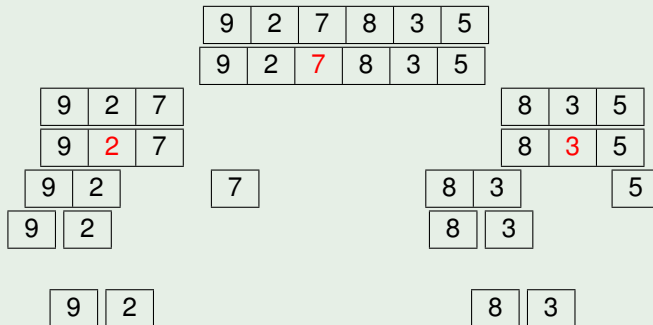
Tri fusion

Illustration du tri par fusion



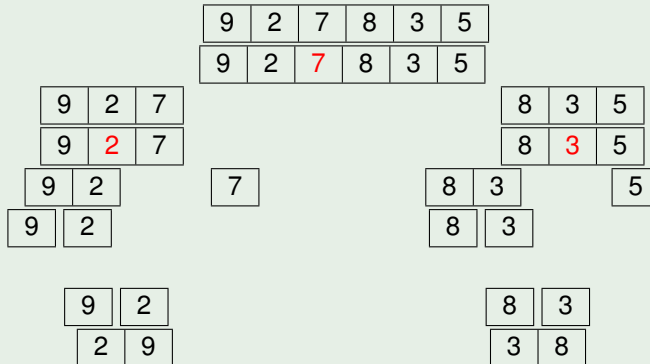
Tri fusion

Illustration du tri par fusion



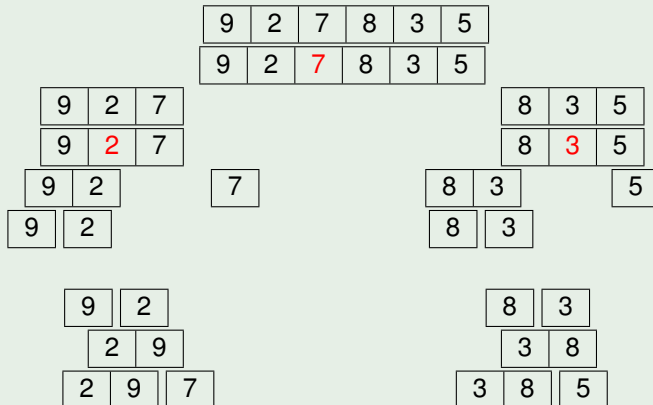
Tri fusion

Illustration du tri par fusion



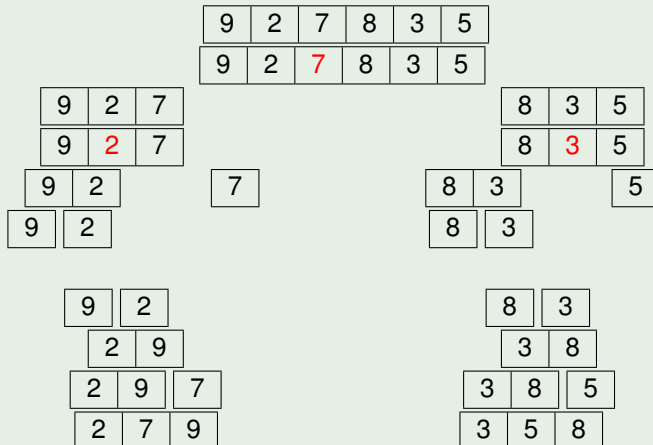
Tri fusion

Illustration du tri par fusion



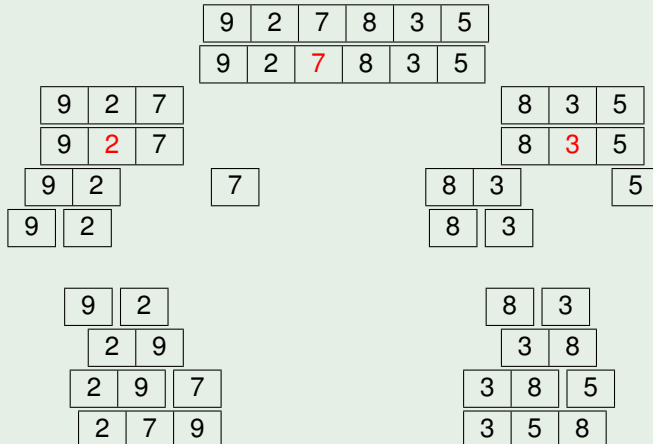
Tri fusion

Illustration du tri par fusion



Tri fusion

Illustration du tri par fusion



Exemple d'implémentation du tri par fusion

```
Fonction triFusion (T:tab N entiers, deb:Entier, fin:Entier):tab N entier
Déclarations
T1, T2 : tableau N/2 entier;
milieu: Entier;
DEBUT
Si (deb >= fin) alors
Retourner T ;
Sinon
milieu = (deb + fin)/ 2;
T1 = triFusion (T, deb, milieu) ;
T2 = triFusion (T, milieu+1, fin)) ;
Retourner fusion (T1, T2) ;
Fin si
FIN
```

Exemple d'implémentation de la fonction de fusion

```
Fonction fusion(T1:tab N entiers, T2:tab M entiers):tab M+N entiers
Déclarations
T: tableau 1..M+N d'entiers; i, j: Entier;
DEBUT
i<--1; j<--1;
Tant que (i+j-1 <> N+M) faire
Si (i<=N) alors
si (j<=M) alors
si (T1[i]<T2[j]) alors T[i+j-1]<--T1[i]; i<--i+1;
sinon T[i+j-1]<--T2[j]; j<--j+1;
Fin si
sinon T[i+j-1]<--T1[i]; i<--i+1;
Fin si
sinon T[i+j-1]<--T2[j]; j<--j+1;
Fin si
Fin Tant que
Retourner T;
```

Calcul de la complexité

Comme Diviser et régner

- Diviser : calcul du milieu de $[debut, fin]$ en $O(1)$;
- Régner : résolution récursive de 2 sous-problèmes de tailles $\frac{n}{2}$ en $2T(\frac{n}{2})$;
- Combiner : fusion en un tableau solution de taille n , en $O(n)$.

On a $T(n) = 2T(\frac{n}{2}) + O(n)$ or d'après le Théorème de résolution de la récurrence :

- si $a > b^k$ alors $T(n) = O(n^{\log_b a})$
- si $a = b^k$ alors $T(n) = O(n^k \log_b n)$
- si $a < b^k$ alors $T(n) = O(f(n)) = O(n^k)$

Comme $a = b = 2$, et $k = 1$ on a $a = b^k$ et donc $T(n) = O(n \log_2(n))$

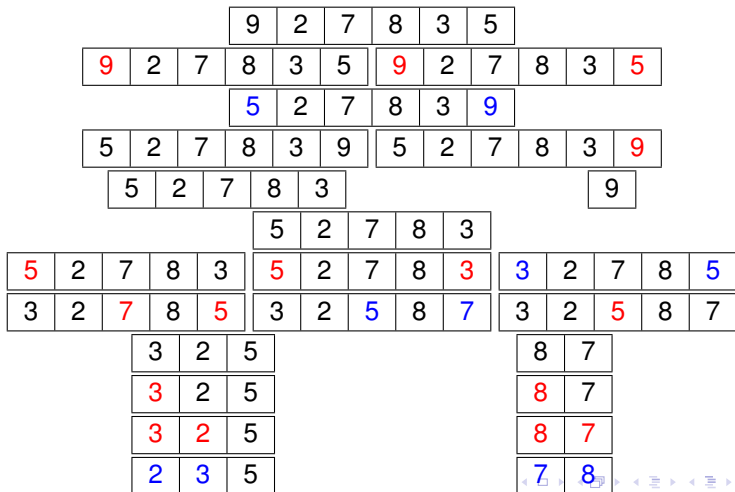
Tri rapide/quicksort

Principe

Il est basé sur la stratégie du *diviser et régner* et se décompose en trois étapes :

- **Diviser** : le tableau $T[\text{début}..\text{fin}]$ est partitionné en 2 sous-tableaux non vides, $T[\text{début}..\text{pivot}]$ et $T[\text{pivot}+1..\text{fin}]$ tels que chaque élément de $T[\text{début}..\text{pivot}]$ soit inférieur ou égal à chaque élément de $T[\text{pivot}+1..\text{fin}]$;
- **pivot** est calculé pendant le partitionnement ;
- **Régner** : les deux sous-tableaux $T[\text{début}..\text{pivot}]$ et $T[\text{pivot}+1..\text{fin}]$ sont triés par des appels récursifs.
- **Combiner** : comme les sous-tableaux sont triés sur place, le tableau $T[\text{début}..\text{fin}]$ est déjà trié à la fin.

Tri rapide/quicksort



Exemple d'implémentation du tri par insertion

Fonction Partionner (T, deb, fin):Entier

Déclarations

x,i,j:Entier

Début

x=T[deb]; i=deb-1; j=fin+1;

Tant que (1)

Répéter {j=j-1} Jusqu'à (T[j]<=x);

Répéter {i=i+1} Jusqu'à (T[i]>=x);

si (i<j) alors

permuter (T[i],T[j]);

sinon retourner j; Fin Si

Fin Tant que

Fin

Procédure TriRapide (T,deb,fin)

Déclarations

pivot:Entier

Début

Si (deb<fin) alors

pivot=Partionner(T,deb,fin);

TriRapide(T,deb,pivot);

TriRapide(T,pivot+1,fin);

Fin si

Fin

Calcul de la complexité

Au pire des cas

Le pire cas correspond au cas où le partitionnement produit une région à $n - 1$ éléments et une à 1 élément. Dans ce cas, le partitionnement coûte $O(n)$ et la récurrence pour le temps d'exécution est alors $T(n) = T(n - 1) + O(n)$ d'où $T(n) = O(n^2)$

Au mieux des cas

Le meilleur cas correspond au cas où le partitionnement produit 2 régions à $\frac{n}{2}$ éléments. Dans ce cas, la récurrence est $T(n) = T(\frac{n}{2}) + O(n)$ d'où $T(n) = O(n \log_2(n))$

Problèmes P/NP et algorithmique avancée

- 1 Généralités
- 2 Classification des problèmes
- 3 Pistes de solution
- 4 Applications du Diviser pour régner
- 5 Applications des algorithmes gloutons**
 - Problème du choix d'activités
 - Autres algorithmes gloutons
- 6 Applications de la programmation dynamique

Généralités

Qu'est-ce qu'un algorithme glouton

C'est une approche de résolution de problèmes d'optimisation qui crée au fur et à mesure une solution optimale et qui se caractérise par :

- le meilleur choix local à chaque étape ;
- le non retour en arrière ;
- la progression descendante qui consiste à faire le choix et à s'attaquer à un problème réduit.

Remarque non superflue

Un algorithme glouton ne conduit pas toujours à la solution optimale. Quand c'est le cas on parlera d'**algorithme glouton exact** sinon d'**heuristiques gloutonnes**.

Problème du choix d'activités

Définition du problème

Considérons n activités (cours) concurrentes numérotées par $\mathcal{S} = \{1, 2, \dots, n\}$ qui doivent utiliser une même et seule ressource (salle de cours) qui ne peut être utilisée que pour une activité à la fois.

- L'activité numérotée i peut être caractérisée par un début noté d_i et une fin notée f_i avec $d_i \leq f_i$. On dit aussi que l'activité i a lieu pendant l'intervalle de temps $[d_i, f_i[$.
- Deux activités i et j sont compatibles si leurs intervalles $[d_i, f_i[$ et $[d_j, f_j[$ ne s'intersectent pas, c'est à dire que $d_i \geq f_j$ ou $d_j \geq f_i$.

Le problème du choix d'activité consiste à déterminer le plus grand nombre d'activités compatibles entre elles.

Problème du choix d'activités

Ordonnancement

Soit $E = \{\delta_1, \delta_2, \dots, \delta_n\}$ un ensemble d'intervalles qui définit pour chaque i , $\delta_i = [a, b[$ avec a et b deux réels tel que $a < b$. On dit que Θ est un ordonnancement de E si et seulement si :

- $\Theta \subseteq E$
- $\forall \delta_a, \delta_b$ avec $\delta_a \neq \delta_b$ on $\delta_a \cap \delta_b = \emptyset$.

Exercice d'application

On pose

$$E = \{[0, 3[, [2, 4[, [3, 6[, [5, 8[, [7, 10[\}$$

Proposez les ordonnancements possibles sur E .

Problème du choix d'activités

Algorithme d'ordonnancement naïf

Fonction Ordonnancement(E : Ensemble d'intervalles δ_i) : Ensemble d'intervalles

Déclarations

Θ : Ensemble d'intervalles

Debut

$\Theta = \emptyset$

Pour tout $\delta_i \in E$ Faire

Si (Pour tout $\delta_j \in \Theta$, on a $\delta_i \cap \delta_j = \emptyset$) Alors

$\Theta = \Theta \cup \{\delta_i\}$

Fin Si

Fin Faire

Retourner Θ

Fin

Exercice d'application

On pose $E = \{[0, 6[, [1, 4[, [2, 3[, [3, 4[, [4, 6[, [5, 10[\}$. Appliquer l'algorithme sur E . La solution est-elle optimale ?

Problème du choix d'activités

Algorithme d'ordonnancement Petite Taille Avant PTA

Fonction Ordonnancement(E : Ensemble d'intervalles δ_i triées dans l'ordre croissant de leurs durées) : Ensemble d'intervalles

Déclarations

Θ : Ensemble d'intervalles

Debut

$\Theta = \emptyset$

Pour tout $\delta_i \in E$ Faire

Si (Pour tout $\delta_j \in \Theta$, on a $\delta_i \cap \delta_j = \emptyset$) Alors

$\Theta = \Theta \cup \{\delta_i\}$

Fin Si

Fin Faire

Retourner Θ

Fin

On pose $E = \{[0, 6[, [1, 4[, [2, 3[, [3, 4[, [4, 6[, [5, 10[\}$. Trier E et appliquer l'algorithme sur E . La solution est-elle optimale ?

Problème du choix d'activités

Contre-Exemple

On pose $E = \{[0, 3[, [2, 4[, [3, 6[, [6, 9[, [8, 10[, [9, 12[\}$ Trier E et appliquer l'algorithme sur E .

Propriétés de l'algorithme PTA

- On pose θ^* comme l'ordonnancement optimal de E ;
- On pose θ comme l'ordonnancement renvoyé par l'algorithme PTA ;

On a alors $\forall E, 2|\theta| \geq |\theta^*|$

Problème du choix d'activités

Algorithme glouton

Fonction OrdonnancementGlouton(S : Ensemble activités, f : liste croissante de fins) :

Ensemble activités

Déclarations

A : Ensemble activités

n, j : Entier

Debut

$n = \text{longueur}(S)$; $A = \{1\}$; $j = 1$

Pour tout i de 2 à n Faire

Si ($d_i \geq f_j$) Alors

$A = A \cup \{i\}$

$j = i$

Fin Si

Fin Faire

Retourner A

Fin

Problème du choix d'activités

Exercice d'application

i	1	2	3	4	5	6	7	8	9	10	11
d	1	3	0	5	3	5	6	8	8	2	12
f	4	5	6	7	8	9	10	11	12	13	13

Dérouler l'algorithme sur cet exemple.

Algorithme glouton

- On trie les activités par ordre croissant de fin : $f_1 \leq f_2 \leq \dots \leq f_n$;
- L'algorithme est très efficace car elle peut déterminer l'ordonnancement en $\Theta(n)$ et il s'agit toujours de la solution optimale.

Problème du monnayeur

Définition du problème

On veut rendre la monnaie avec le moins de pièces possibles :

- Un commerçant aguerri rendra toujours la plus grosse pièce qui peut sortir de ce qui reste ;
- Cette approche intuitive est la méthode la plus efficace puisqu'à chaque étape on fait le choix qui minimise ce qui reste à rendre ;
- Néanmoins cela dépend du système monétaire.

Cas du Shilling

Pour une livre anglaise valant 20 shillings et le shilling ayant 4 subdivisions : le penny ($\frac{1}{12}$), et 3 pièces valant 3, 4 et 6 pence (pluriel penny), rendre 8 pence conduirait à rendre une pièce de 6 pence et 2 pièces de 1 penny au lieu de rendre 2 pièces de 4 pence.

Problème du monnayeur

Algorithme glouton

- Il y a donc des cas où l'algorithme glouton ne marche pas ;
- Mais ils se révèlent très efficaces dans de nombreux contextes.

Caractéristiques de problèmes solvables

Les algorithmes gloutons résolvent bien les problèmes ayant les caractéristiques suivantes :

- 1 **Propriété du choix glouton** : on peut arriver à une solution optimale en effectuant un choix localement optimal ;
- 2 **Sous-structure optimale** : une solution optimale du problème fait apparaître une solution optimale à un sous-problème.

Problème du sac à dos

Définition du problème

On vous met dans une cache de trésor contenant n objets :

- chaque objet i a une valeur v_i et une masse m_i (tous entiers) ;
- vous devez prendre un butin de valeur maximale mais vous ne pouvez pas prendre plus d'une masse maximale M (entier) ;
- Quels objets allez-vous prendre pour maximiser le butin ?

Deux variantes possibles

- **Tout Ou Rien TOR** : chaque objet est pris ou pas, on ne peut pas prendre une partie d'un objet et un objet ne peut pas être pris plus d'une fois ;
- **Fractionnaire FRAC** : on peut prendre des parties d'objets.

Problème du sac à dos

Stratégie gloutonne

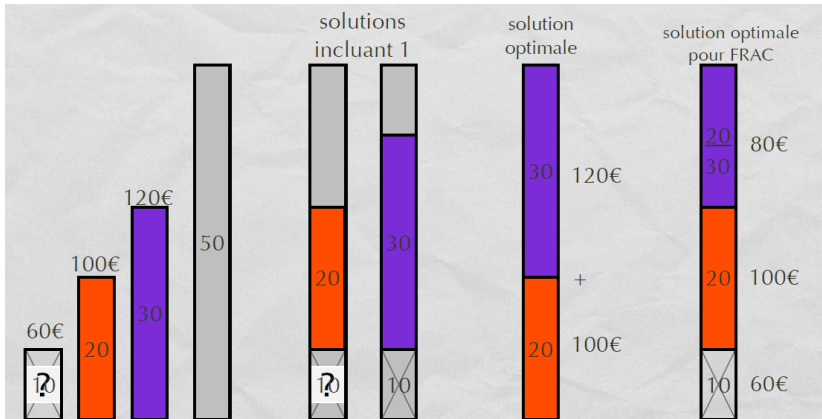
- calculer la valeur par kilo de chaque objet ;
- prendre le plus possible l'objet ayant la plus grande valeur par kilo et ainsi de suite jusqu'à remplir le sac ;
- Si les objets sont triés, la complexité est de $O(n \log n)$;
- Cela marche pour FRAC mais pas pour TOR.

Illustration

Votre sac peut prendre 50kgs et dans la cache de trésor on a :

- un objet de 10 kgs de valeur 60 euros ;
- un objet de 20 kgs de valeur 100 euros ;
- un objet de 30 kgs de valeur 120 euros ;

Problème du sac à dos



Problème du sac à dos

Sous-structure optimale

Les 2 variantes ont cette caractéristique :

- TOR : le chargement optimal pesant M , si on retire un objet j , le chargement restant est le meilleur que l'on puisse prendre si la masse maximale était $M - m_j$ à partir des $n - 1$ objets restant, l'objet j étant exclu ;
- FRAC : si on retire le poids m dans le chargement optimal, le reste du chargement est le meilleur que l'on puisse prendre pour une masse de $M - m$ à partir des $n - 1$ objets restant et des $m_j - m$ kilos de l'objet j .

- 1 Généralités
- 2 Classification des problèmes
- 3 Pistes de solution
- 4 Applications du Diviser pour régner
- 5 Applications des algorithmes gloutons
- 6 Applications de la programmation dynamique**
 - Principe
 - Quelques algorithmes de programmation dynamique

Généralités

Limites du "Diviser Régner"

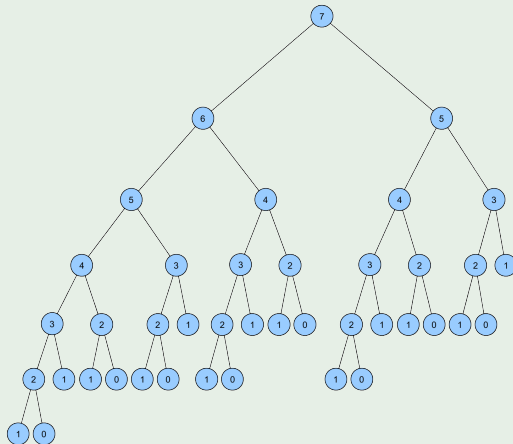
Dans le paradigme "Diviser Régner" on peut rencontrer plusieurs fois le même sous-problème et le traiter ainsi plusieurs fois.

Illustration

Soit la suite des appels récursifs faits au calcul de **FibRecursive(7)** :

- $\text{FibRecursive}(7) = \text{FibRecursive}(6) + \text{FibRecursive}(5)$;
- $\text{FibRecursive}(6) = \text{FibRecursive}(5) + \text{FibRecursive}(4)$;
- $\text{FibRecursive}(5) = \text{FibRecursive}(4) + \text{FibRecursive}(3)$;
- $\text{FibRecursive}(4) = \text{FibRecursive}(3) + \text{FibRecursive}(2)$;
- $\text{FibRecursive}(3) = \text{FibRecursive}(2) + \text{FibRecursive}(1)$;
- $\text{FibRecursive}(2) = \text{FibRecursive}(1) + \text{FibRecursive}(0)$.

Généralités



Généralités

Programmation dynamique

Pour éviter de résoudre les mêmes sous-problèmes, il suffit de mémoriser les résultats des sous-problèmes dans un tableau ou une liste. C'est cette technique qu'on appelle programmation **dynamique**.

Types de programmation dynamique

- **Type récursif "Top down" dite de mémorisation** : on utilise toujours la récurrence, mais lors d'un appel récursif, avant d'effectuer un calcul on regarde dans le tableau si on ne l'a pas déjà effectué.
- **Type itératif "Bottom Up"** : on résout les sous-problèmes des plus petits aux plus grands. Au fur et à mesure on stocke les résultats obtenus dans le tableau jusqu'à la taille voulue.

Généralités

Exercice d'application

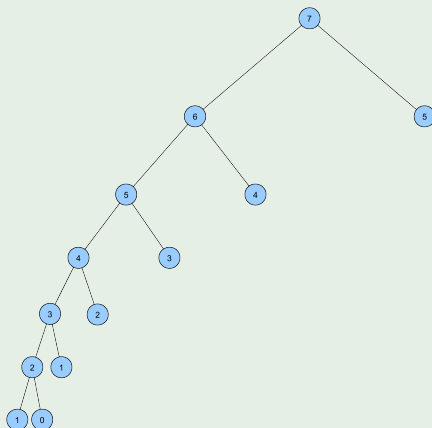
On pose la suite Fibonacci comme suit :

$$Fib(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ Fib(n-1) + Fib(n-2) & \text{si } n \geq 2 \end{cases}$$

- Proposer une fonction récursive naïve qui calcule $Fib(n)$.
- Utiliser la programmation dynamique **Top Down** pour proposer une fonction qui calcule $Fib(n)$.
- Utiliser la programmation dynamique **Bottom Up** pour proposer une fonction qui calcule $Fib(n)$.

Généralités

Illustration Programmation Dynamique "Top Down"



Généralités

Différence entre les types de programmation dynamique

Dans un algorithme **Bottom Up** on résout tous les sous-problèmes de taille inférieure, alors que dans un algorithme **Top Down** on ne résout que ceux nécessaires.

Différence entre programmation dynamique et approche gloutonne

- Dans la programmation dynamique on calcule les solutions de tous les sous-problèmes et on les combine de façon optimale.
- Dans un algorithme glouton on choisit à tout moment ce qui semble être la meilleure solution et on résout les sous-problèmes qui en découlent.

Exemples d'algorithmes de programmation dynamique

Monayeur

- **Problème** : On a un montant M et des pièces de valeurs $v_1, v_2, \dots, v_n$. On veut trouver des entiers x_i tels que : $\sum_{i=1}^n x_i v_i = M$ en minimisant $\sum_{i=1}^n x_i$.
- **Solution** : construire une table de programmation dynamique $C[1\dots n][0\dots M]$ où $C[i][j]$ est le nombre minimum de pièces pour produire exactement le montant j en utilisant seulement des pièces de valeurs $v_1, v_2, \dots, v_i$.
- La solution optimale est en $C[n][M]$.

Exemples d'algorithmes de programmation dynamique

Pistes de solution

On peut poser :

$$\begin{cases} C[i][0] = 0 \\ C[i][j] = \min(C[i-1][j], 1 + C[i][j - v_i]) \end{cases}$$

Algorithme

- Si on veut rendre un montant $M = 8$ et qu'on a des pièces de valeurs $v_1 = 1$, $v_2 = 4$ et $v_3 = 6$, proposer la table de programmation dynamique $C[1...3][0...8]$.
- Proposer l'algorithme pour déterminer le nombre de pièces.
- Quelles remarques ?